

HDOC 34

Name: Keshav Mittal

SAP ID: 590032934

Batch: 16

1. Algorithm

1. Start main program.
2. Display the menu with options 1 to 3.
3. Read the user's choice and use a switch statement.
4. Case 1 (Left Distributive Property):
 - Read dimensions m, n, p . Declare matrices $A(m \times n)$, $B(n \times p)$, $C(n \times p)$.
 - Read elements for matrices A , B , and C .
 - Compute $B + C$, then multiply by A to get $LHS = A * (B + C)$.
 - Compute $A * B$ and $A * C$, then add them to get $RHS = (A * B) + (A * C)$.
 - Compare LHS and RHS matrices. If equal, output that the left distributive property is verified.
5. Case 2 (Right Distributive Property):
 - Read dimensions m, n, p . Declare matrices $A(m \times n)$, $B(m \times n)$, $C(n \times p)$.
 - Read elements for matrices A , B , and C .
 - Compute $A + B$, then multiply by C to get $LHS = (A + B) * C$.
 - Compute $A * C$ and $B * C$, then add them to get $RHS = (A * C) + (B * C)$.
 - Compare LHS and RHS matrices. If equal, output that the right distributive property is verified.
6. Case 3 (Exit): Terminate the program.
7. Stop

2. Test-Case Table

TC ID	Scenario	Inputs	Expected Output
TC-01	Left Distributive	Choice: 1. $m=2, n=2, p=2$. $A=[1 \ 1; 1 \ 1]$, $B=[2 \ 2; 2 \ 2]$, $C=[1 \ 1; 1 \ 1]$	$A*(B+C) = [6 \ 6; 6 \ 6]$, $(A*B)+(A*C) = [6 \ 6; 6 \ 6]$. Verified.
TC-02	Right Distributive	Choice: 2. $m=2, n=2, p=2$. $A=[1 \ 0; 0 \ 1]$, $B=[0 \ 1; 1 \ 0]$, $C=[2 \ 2; 2 \ 2]$	$(A+B)*C = [4 \ 4; 4 \ 4]$, $(A*C)+(B*C) = [4 \ 4; 4 \ 4]$. Verified.
TC-03	Exit	Choice: 3	Exiting program...

3. C Program Source Code

```
#include <stdio.h>

void readMatrix(int r, int c, int mat[r][c]); void printMatrix(int r, int
c, int mat[r][c]);
void addMatrices(int r, int c, int m1[r][c], int m2[r][c], int res[r][c]);
void multiplyMatrices(int r1, int c1, int c2, int m1[r1][c1], int m2[c1][c2], int res[r1][c2]);
int compareMatrices(int r, int c, int m1[r][c], int m2[r][c]); void verifyLeftDistributive(void);
void verifyRightDistributive(void);

int main() {
    int choice; do {
        printf("\n--- Matrix Distributive Properties Menu ---\n");
        printf("1. Verify Left Distributive Property:  $A*(B+C) = (A*B) + (A*C)$ \n"); printf("2. Verify Right Distributive Property:
 $(A+B)*C = (A*C) + (B*C)$ \n"); printf("3. Exit\n");
        printf("Enter your choice: "); scanf("%d", &choice);

        switch (choice) { case 1:
            verifyLeftDistributive(); break;
            case 2:
            verifyRightDistributive(); break;
            case 3:
            printf("Exiting program...\n"); break;
            default:
            printf("Error: Invalid choice.\n");
        }
    } while (choice != 3);

    return 0;
}

void readMatrix(int r, int c, int mat[r][c]) { for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) { scanf("%d", &mat[i][j]);
    }
}
}

void printMatrix(int r, int c, int mat[r][c]) { for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) { printf("%d\t", mat[i][j]);
    }
    printf("\n");
}
```

```

}

void addMatrices(int r, int c, int m1[r][c], int m2[r][c], int res[r][c]) { for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) { res[i][j] = m1[i][j] +
        m2[i][j];
    }
}
}

```

```

void multiplyMatrices(int r1, int c1, int c2, int m1[r1][c1], int m2[c1][c2], int res[r1][c2]) {
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++) { res[i][j] = 0;
            for (int k = 0; k < c1; k++) { res[i][j] += m1[i][k] *
                m2[k][j];
            }
        }
    }
}

```

```

int compareMatrices(int r, int c, int m1[r][c], int m2[r][c]) { for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) { if (m1[i][j] !=
        m2[i][j]) {
        return 0;
    }
}
}
return 1;
}

```

```

void verifyLeftDistributive(void) { int m, n, p;
    printf("Enter dimensions m, n, p: "); scanf("%d %d %d", &m,
        &n, &p);
    if (m <= 0 || n <= 0 || p <= 0) return;

    int A[m][n], B[n][p], C[n][p]; int
    B_plus_C[n][p], LHS[m][p];
    int AB[m][p], AC[m][p], RHS[m][p];

    printf("Enter elements of Matrix A (%dx%d):\n", m, n); readMatrix(m, n, A);
    printf("Enter elements of Matrix B (%dx%d):\n", n, p); readMatrix(n, p, B);
    printf("Enter elements of Matrix C (%dx%d):\n", n, p); readMatrix(n, p, C);

    addMatrices(n, p, B, C, B_plus_C); multiplyMatrices(m, n, p, A,
    B_plus_C, LHS);

    multiplyMatrices(m, n, p, A, B, AB);
    multiplyMatrices(m, n, p, A, C, AC); addMatrices(m,
    p, AB, AC, RHS);

    printf("\nA * (B + C):\n");
}

```

```

printMatrix(m, p, LHS);

printf("\n(A * B) + (A * C):\n"); printMatrix(m, p, RHS);

if(compareMatrices(m, p, LHS, RHS)) {
    printf("\nSince  $A*(B+C) = (A*B)+(A*C)$ , the left distributive property is verified.
\n");
} else {
    printf("\nMatrices do not match.\n");
}
}

void verifyRightDistributive(void) { int m, n, p;
    printf("Enter dimensions m, n, p: "); scanf("%d %d %d", &m,
    &n, &p);
    if (m <= 0 || n <= 0 || p <= 0) return;

    int A[m][n], B[m][n], C[n][p]; int
    A_plus_B[m][n], LHS[m][p];
    int AC[m][p], BC[m][p], RHS[m][p];

    printf("Enter elements of Matrix A (%dx%d):\n", m, n); readMatrix(m, n, A);
    printf("Enter elements of Matrix B (%dx%d):\n", m, n); readMatrix(m, n, B);
    printf("Enter elements of Matrix C (%dx%d):\n", n, p); readMatrix(n, p, C);

    addMatrices(m, n, A, B, A_plus_B); multiplyMatrices(m, n, p,
    A_plus_B, C, LHS);

    multiplyMatrices(m, n, p, A, C, AC);
    multiplyMatrices(m, n, p, B, C, BC); addMatrices(m,
    p, AC, BC, RHS);

    printf("\n(A + B) * C:\n"); printMatrix(m, p, LHS);

    printf("\n(A * C) + (B * C):\n"); printMatrix(m, p, RHS);

    if(compareMatrices(m, p, LHS, RHS)) {
        printf("\nSince  $(A+B)*C = (A*C)+(B*C)$ , the right distributive property is verified.
\n");
    } else {
        printf("\nMatrices do not match.\n");
    }
}

```

4. Compilation and Execution

```
(kali@kali)-[~/liak]
$ cc hdoc_34.c
```

```
(kali@kali)-[~/liak]
$ ./a.out
```

— Matrix Distributive Properties Menu —

1. Verify Left Distributive Property: $A*(B+C) = (A*B) + (A*C)$
2. Verify Right Distributive Property: $(A+B)*C = (A*C) + (B*C)$
3. Exit

Enter your choice: 1

Enter dimensions m, n, p: 3 4 5

Enter elements of Matrix A (3×4):

2 3 4 5 5 2 4 2 4 5 6 7 8

Enter elements of Matrix B (4×5):

2 3 4 5 6 7 8 9 8 7 6 5 4 3 2 3 4 5 6 7

Enter elements of Matrix C (4×5):

3 4 5 6 7 8 8 5 4 3 2 1 2 3 4 5 6 7 8 9

A * (B + C):

139	127	136	144	152
153	103	111	121	131
227	199	214	226	238

(A * B) + (A * C):

139	127	136	144	152
153	103	111	121	131
227	199	214	226	238

Since $A*(B+C) = (A*B)+(A*C)$, the left distributive property is verified.

— Matrix Distributive Properties Menu —

1. Verify Left Distributive Property: $A*(B+C) = (A*B) + (A*C)$
2. Verify Right Distributive Property: $(A+B)*C = (A*C) + (B*C)$
3. Exit

Enter your choice: Error: Invalid choice.

— Matrix Distributive Properties Menu —

1. Verify Left Distributive Property: $A*(B+C) = (A*B) + (A*C)$
2. Verify Right Distributive Property: $(A+B)*C = (A*C) + (B*C)$
3. Exit

Enter your choice: 3